HƯỚNG DẪN CHI TIẾT THỰC HIỆN BÀI TẬP LỚN

Học phần: Phân tích thiết kế hướng đối tượng (OOAD)

I. TƯ DUY TỔNG THỂ

Bài tập lớn (BTL) không phải là bài tập lập trình thuần túy. Mục tiêu là **"Tư duy mô hình hóa"**. Sản phẩm cuối cùng của bạn phải thể hiện được sự chuyển dịch logic từ:

> **Bài toán trong thế giới thực (Bài toán nghiệp vụ) → Mô hình phân tích (Cái gì?) → Mô hình thiết kế (Như thế nào?) → Mã nguồn (Code).**

Cần tránh tư duy "code trước, vẽ sau". Mọi dòng code phải phản ánh đúng thiết kế đã chứng minh qua biểu đồ UML và các nguyên lý thiết kế. Nên coi báo cáo BTL như một bản hợp đồng kỹ thuật giữa nhóm sinh viên và giảng viên, nơi các bạn chứng minh năng lực phân tích và thiết kế của mình.

II. QUY TRÌNH THỰC HIỆN CHI TIẾT

Bước 1: Thành lập nhóm và Lựa chọn đề tài

- **Quy mô:** Khoảng 3-5 sinh viên/nhóm hoặc theo phân công của giảng viên. Các thành viên trong nhóm nên bầu một nhóm trưởng và phân công vai trò rõ ràng cho từng người.
- **Chọn đề tài:** Ưu tiên các bài toán có nghiệp vụ rõ ràng, dữ liệu phong phú. Tránh chọn đề tài quá mơ hồ hoặc quá phức tạp về thuật toán. **Quan trọng:** Xác định rõ **phạm vi** (scope) của đề tài ngay từ đầu sao cho phù hợp với thời gian thực hiện (thường là 2-3 tuần). Ví dụ: "Quản lý thư viện" có thể chỉ dừng ở mượn/trả sách, hoặc mở rộng sang đặt trước, phí phạt, quản lý bản sao.
- **Khảo sát:** Thực hiện phỏng vấn (thực tế hoặc giả định) hoặc đọc tài liệu để hiểu quy trình nghiệp vụ. Ghi chép lại các vấn đề, các "nỗi đau" (pain points) của hệ thống cũ.

Bước 2: Phân tích Yêu cầu

Đây là giai đoạn tìm hiểu bài toán, xây dựng "từ điển nghiệp vụ".

1. **Xác định Tác nhân (Actors):** Là vai trò (Role), không phải con người cụ thể. Ví dụ: "Thủ thư", "Độc giả", "Quản trị viên". Tác nhân cũng có thể là một hệ thống ngoài (ví dụ: Cổng thanh toán).
2. **Xác định Ca sử dụng (Use Cases):** Dựa trên mục tiêu của tác nhân. Mỗi UC phải mang lại giá trị đo lường được.
    - *Lưu ý:* Không nên vẽ sơ đồ quá "mịn". Ví dụ: "Đăng nhập", "Nhập mật khẩu" thường không phải là UC chính, hãy gộp chúng vào UC "Xác thực người dùng".

3. **Vẽ Use Case Diagram:**
    - Ngoài quan hệ giữa Tác nhân và UC, nên sử dụng: (1) quan hệ <<include>> cho các phần logic chung (ví dụ: "Mượn sách" include "Kiểm tra tình trạng sách") và (2) quan hệ <<extend>> cho các chức năng bổ sung tùy chọn (ví dụ: "Thanh toán" extend "Áp dụng mã giảm giá").
    - Thể hiện rõ Boundary (khung hệ thống) và mối quan hệ giữa các Actor.
4. **Đặc tả Use Case (Use Case Description):** Viết dưới dạng bảng hoặc văn bản. Bao gồm: Tên UC, Tác nhân, Mục tiêu, Tiền điều kiện, Luồng chính (Main Flow), Luồng phụ/ngoại lệ (Alternative/Exception Flow), Hậu điều kiện.
5. **Yêu cầu phi chức năng (NFRs):** Ghi rõ các ràng buộc về hiệu năng, bảo mật, môi trường triển khai, …

Bước 3: Phân tích Đối tượng

Giai đoạn này xác định thực thể (Entity) mà không quan tâm đến UI hay CSDL.

1. **Phân tích danh từ (Noun Analysis):** Gạch chân danh từ trong đặc tả yêu cầu để tìm lớp ứng viên.
    - *Tiêu chí giữ:* Thực thể bền vững, có dữ liệu và hành vi đi kèm.
    - *Tiêu chí loại:* Chi tiết giao diện (Form, Button), chi tiết kỹ thuật (Database, Session), các hành động tức thời không có vòng đời.
2. **Conceptual Model (Class Diagram):** Conceptual Model mô tả các khái niệm quan trọng của miền nghiệp vụ và mối quan hệ giữa chúng, chưa quan tâm đến UI, database hay framework. Lưu ý:
    - Đừng vẽ thuộc tính id (khóa chính) vào mô hình khái niệm – Nó là chi tiết cài đặt.
    - Sử dụng Lớp kết hợp (Association Class) khi mối quan hệ giữa hai lớp cần lưu trữ dữ liệu (ví dụ: Member và Book có lớp Loan để lưu borrowDate, dueDate).
3. **Thẻ CRC (Class-Responsibility-Collaborator):** Đây là một công cụ quan trọng để thực hành Brainstorming. Nó hỗ trợ chuyển Conceptual Model → Sequence Diagram.
    - Phân bổ "Trách nhiệm" (Responsibility) dựa trên **Information Expert** (ai giữ thông tin thì làm).
    - Xác định "Cộng tác viên" (Collaborators) - những lớp cần tương tác để hoàn thành trách nhiệm.
    - *Ví dụ:* Lớp Hire cần tính tiền phạt, nhưng nó không biết giá thuê của xe → nó cần collaborate với lớp Bike.

Bước 4: Thiết kế Hệ thống

Đây là giai đoạn chuyển từ bài toán sang giải pháp phần mềm.

1. **Kiến trúc phân tầng (Layering):**
    - **Presentation Layer (Boundary):** Hiển thị, nhận tương tác.
    - **Application Layer (Control):** Điều phối use case, xử lý logic nghiệp vụ.
    - **Domain Layer (Entity):** Logic cốt lõi, dữ liệu.
    - **Infrastructure Layer (Persistence):** CSDL, kết nối.
2. **Áp dụng BCE (Boundary - Control - Entity):**
    - Tách biệt logic hiển thị, logic điều phối và dữ liệu.
    - *Quy tắc tương tác:* Tác nhân → Boundary → Control → Entity. Entity không được gọi ngược lên Boundary/Control.
    - Lưu ý: BCE là **một mô hình tư duy để phân tách trách nhiệm**, không phải công thức máy móc về số lượng lớp. Khi sang các kiến trúc hiện đại hơn, BCE có thể được ánh xạ sang Controller/Application Service/Domain Object/Repository... chứ không nhất thiết giữ nguyên tên.
3. **Thiết kế chi tiết:**
    - **Sequence Diagram:** Nên sử dụng mô hình BCE. Có thể áp dụng thêm các khung alt, opt, loop, ref. Cần thể hiện rõ thông điệp (message) và đối tượng nhận.
    - **Design Class Diagram (DCD):** Bổ sung phương thức, kiểu dữ liệu, tính điều hướng (Navigability). Đây là sơ đồ "chốt" để code.
    - **Biểu đồ trạng thái (State Machine Diagram):** Dùng cho các đối tượng có vòng đời phức tạp (ví dụ: Order: Chờ thanh toán → Đã thanh toán → Đang giao → Đã nhận → Hoàn trả).
4. **Một số nguyên tắc thiết kế:**
    - **Information Expert:** Tại sao gán trách nhiệm này cho lớp X? (Vì X có đủ dữ liệu).
    - **Creator:** Tại sao lớp X tạo ra lớp Y? (Vì X chứa hoặc quản lý Y).
    - **Controller:** Tại sao chọn lớp này làm Controller? (Vì nó đại diện cho hệ thống hoặc use case).
    - **Low Coupling / High Cohesion:** Tại sao tách lớp này ra? (Để giảm phụ thuộc, tăng tính tập trung).

Bước 5: Thiết kế Cơ sở dữ liệu

1. **Chuyển đổi từ Class Diagram sang ERD:**
    - Class → Table.
    - Attribute → Column.
    - Association 1-n → Thêm khóa ngoại vào bảng con (đầu 'n').
    - Association n-m → Tạo bảng trung gian.
    - Inheritance → Chọn chiến lược (Single Table hoặc Concrete Table).
    - Lưu ý: Không phải mọi class đều trở thành table và không phải mọi association trong class diagram đều ánh xạ trực tiếp thành khóa ngoại (foreign key).
2. **Viết Script SQL:** Tạo bảng, khóa chính, khóa ngoại, ràng buộc (Check, Unique).
3. **Dữ liệu mẫu (Seed Data):** Tạo sẵn dữ liệu để kiểm thử.

Bước 6: Cài đặt và Kiểm thử (Chương 4 – khuyến khích, không bắt buộc)

1. **Mapping UML to Code:** Chuyển đổi DCD thành code. Lưu ý: Các thuộc tính private, methods public/protected phải khớp với sơ đồ.
2. **Handling Errors:** Áp dụng nguyên lý SOLID để code dễ bảo trì (đặc biệt là Dependency Inversion - dùng Interface/Abstract class để giảm coupling).
3. **Kiểm thử:** Viết các Test Case cho các luồng chính và luồng ngoại lệ. Ghi lại các kết quả thực tế.

III. HƯỚNG DẪN ÁP DỤNG CÁC NGUYÊN LÝ THIẾT KẾ

1. Nguyên lý GRASP:

- **Information Expert:** "Ai có đủ dữ liệu để trả lời câu hỏi này?". Đừng bắt lớp Controller đi hỏi dữ liệu của Entity rồi tính toán. Hãy để Entity tự tính toán.
    - *Ví dụ:* Loan có borrowDate và period → Loan tự tính calculateDueDate().
- **Creator:** Ai sở hữu lớp đó thì lớp đó tạo ra nó.
    - *Ví dụ:* Sale chứa SalesLineItem → Sale có trách nhiệm tạo SalesLineItem.
- **Controller:** Giao diện (Boundary) không được chứa logic nghiệp vụ. Chỉ gọi phương thức của lớp Control.
    - *Ví dụ:* BorrowForm gọi LoanController.borrowBook().

- **High Cohesion:** Lớp chỉ nên làm một nhóm việc liên quan đến nhau. Nếu lớp Customer vừa quản lý thông tin, vừa in hóa đơn, vừa gửi email → Vi phạm (Cần tách lớp).
- **Low Coupling:** Giảm sự phụ thuộc bằng cách dùng Interface hoặc thông qua lớp trung gian.

2. Nguyên lý SOLID

- **SRP (Single Responsibility):** Mỗi lớp chỉ nên thay đổi vì một lý do duy nhất.
- **OCP (Open/Closed):** Mở rộng tính năng mới mà không phải sửa code cũ (dùng đa hình/interface).
- **LSP (Liskov Substitution):** Lớp con phải thay thế được lớp cha mà không làm hỏng hệ thống.
- **ISP (Interface Segregation):** Tránh các interface quá lớn.
- **DIP (Dependency Inversion):** Lớp cấp cao không phụ thuộc lớp cấp thấp, cả hai cùng phụ thuộc abstraction.

IV. CẤU TRÚC BÁO CÁO

Báo cáo được trình bày theo trình tự tiến hóa của hệ thống. Riêng Chương 4 (Cài đặt và Kiểm thử) không bắt buộc nhưng khuyến khích sinh viên thực hiện.

Chương 1: Phân tích yêu cầu

- Giới thiệu bài toán.
- Liệt kê Use Case (High-level & Expanded format).
- Biểu đồ Use Case (kèm quan hệ Include/Extend).
- Mô tả chi tiết các tác nhân (Actors).

Chương 2: Phân tích đối tượng

- Mô hình khái niệm (Conceptual Model).
- Thẻ CRC (Bảng hoặc file ảnh).
- Biểu đồ tuần tự hệ thống (System Sequence Diagram - SSD).

Chương 3: Thiết kế

- Kiến trúc tầng (Layering diagram).
- Sơ đồ các lớp BCE.

- Sequence Diagram (Cho 2-3 ca sử dụng phức tạp).
- Design Class Diagram (DCD): Phải có đầy đủ phương thức và thuộc tính.
- Biểu đồ trạng thái (Statechart) cho các thực thể vòng đời phức tạp.
- Phân tích GRASP/SOLID: Mỗi lớp quan trọng nên có giải thích: "Tại sao chọn Controller này? Tại sao lớp này lại có High Cohesion?"…

Chương 4: Cài đặt và Kiểm thử

- Cấu trúc project (Folder tree).
- Ảnh chụp màn hình (UI) tương ứng với Use Case.
- Kết quả Test Case.

V. GỢI Ý ĐỀ TÀI VÀ PHẠM VI YÊU CẦU

5.1. Nguyên tắc lựa chọn và thực hiện đề tài

Bài tập lớn yêu cầu nhóm vận dụng các kiến thức của học phần Phân tích và thiết kế hướng đối tượng để phân tích và thiết kế một hệ thống phần mềm từ một bài toán nghiệp vụ thực tế.

Mục tiêu chính của bài tập lớn không phải là viết được nhiều mã nguồn, mà là thể hiện được khả năng:

- hiểu và mô tả đúng bài toán nghiệp vụ;
- xác định được các nhu cầu của người sử dụng;
- xây dựng mô hình phân tích phù hợp;
- xác định các đối tượng và trách nhiệm của chúng;
- thiết kế sự tương tác giữa các đối tượng;
- chuyển từ mô hình phân tích sang thiết kế;
- giải thích và bảo vệ được các quyết định thiết kế của nhóm.

Các đề tài dưới đây chỉ mô tả **bối cảnh và phạm vi nghiệp vụ**. Nhóm phải tự phân tích để xác định các thành phần cần thiết của hệ thống.

Lưu ý: Không có một lời giải UML duy nhất cho mỗi đề tài. Hai nhóm có thể đưa ra các mô hình khác nhau nhưng vẫn được đánh giá tốt nếu mô hình phù hợp với nghiệp vụ, nhất quán và giải thích được các quyết định thiết kế.

Nhóm được phép bổ sung hoặc điều chỉnh một số chi tiết nghiệp vụ để làm cho bài toán hợp lý hơn, nhưng phải:

1. nêu rõ các giả định đã đưa ra;
2. bảo đảm các mô hình trong toàn bộ báo cáo nhất quán với các giả định đó;
3. không mở rộng phạm vi quá lớn so với thời gian thực hiện bài tập.

5.2. Các đề tài gợi ý

Đề tài 1. Hệ thống quản lý thư viện

Một thư viện phục vụ sinh viên và giảng viên trong trường đại học đang muốn tin học hóa hoạt động quản lý tài liệu và phục vụ bạn đọc.

Hiện nay thư viện quản lý thông tin tài liệu, bạn đọc và việc mượn trả còn thực hiện bằng nhiều phương thức khác nhau. Thư viện cần một hệ thống hỗ trợ việc tìm kiếm tài liệu, theo dõi tình trạng tài liệu, phục vụ mượn – trả và quản lý các vấn đề phát sinh trong quá trình sử dụng tài liệu.

Hệ thống cần đáp ứng hoạt động của thư viện từ khi tài liệu được đưa vào quản lý cho đến khi được sử dụng, trả lại hoặc xử lý các trường hợp bất thường.

Phạm vi: tập trung vào hoạt động nghiệp vụ của thư viện và người sử dụng thư viện; không yêu cầu xây dựng hệ thống quản lý toàn bộ hoạt động của trường đại học.

Đề tài 2. Hệ thống quản lý sinh viên

Một trường đại học cần xây dựng hệ thống hỗ trợ quản lý sinh viên trong quá trình học tập.

Hệ thống được sử dụng để quản lý thông tin sinh viên, lớp học, môn học và kết quả học tập. Trong quá trình học, sinh viên có thể đăng ký học các môn, tham gia các lớp học phần và nhận kết quả. Nhà trường cần theo dõi tình trạng học tập và cung cấp thông tin cần thiết cho các bên liên quan.

Trong thực tế có thể phát sinh các trường hợp như thay đổi đăng ký học, hủy đăng ký, cập nhật kết quả hoặc xử lý những trường hợp không đáp ứng điều kiện đăng ký.

Phạm vi: tập trung vào các hoạt động quản lý học tập của sinh viên; không cần xây dựng toàn bộ hệ thống quản trị đại học.

Đề tài 3. Hệ thống quản lý khách sạn

Một khách sạn muốn xây dựng hệ thống hỗ trợ hoạt động đặt phòng và phục vụ khách.

Khách có thể tìm kiếm phòng phù hợp, đặt phòng và sử dụng dịch vụ trong thời gian lưu trú. Nhân viên khách sạn cần quản lý tình trạng phòng, thông tin khách và các hoạt động liên quan đến việc nhận phòng, trả phòng và thanh toán.

Trong quá trình hoạt động có thể xảy ra các trường hợp như khách thay đổi hoặc hủy đặt phòng, phòng không còn phù hợp với đặt phòng ban đầu, khách sử dụng thêm dịch vụ hoặc phát sinh các khoản cần thanh toán.

Phạm vi: tập trung vào quản lý phòng, đặt phòng, lưu trú, dịch vụ và thanh toán; không yêu cầu quản lý toàn bộ hoạt động nhân sự hoặc tài chính của khách sạn.

Đề tài 4. Hệ thống quản lý phòng tập thể thao

Một trung tâm thể thao có nhiều hội viên sử dụng các dịch vụ và khu vực tập luyện khác nhau.

Trung tâm cần quản lý thông tin hội viên, các gói dịch vụ, thời hạn sử dụng và tình trạng tham gia của hội viên. Hội viên có thể đăng ký hoặc gia hạn dịch vụ và sử dụng các tiện ích của trung tâm theo điều kiện tương ứng.

Nhân viên trung tâm cần theo dõi việc đăng ký, thanh toán, sử dụng dịch vụ và xử lý những trường hợp như hết hạn, thay đổi gói dịch vụ hoặc vi phạm điều kiện sử dụng.

Phạm vi: tập trung vào quản lý hội viên và các dịch vụ của trung tâm; không cần mô hình hóa chi tiết hệ thống kế toán hoặc quản lý nhân sự.

Đề tài 5. Hệ thống quản lý phòng khám

Một phòng khám cần hệ thống hỗ trợ quản lý quá trình tiếp nhận và khám chữa bệnh cho bệnh nhân.

Bệnh nhân có thể đăng ký khám và cung cấp thông tin cần thiết. Trong quá trình khám, bác sĩ cần xem thông tin liên quan, ghi nhận kết quả khám và đưa ra hướng xử lý. Phòng khám cũng cần quản lý lịch khám và các khoản chi phí liên quan.

Một bệnh nhân có thể đến khám nhiều lần và mỗi lần khám có thể phát sinh những thông tin khác nhau. Trong thực tế cũng có thể xảy ra các trường hợp thay đổi lịch, hủy lịch hoặc bệnh nhân không đến theo lịch đã đăng ký.

Phạm vi: tập trung vào quy trình tiếp nhận – khám – thanh toán của phòng khám. Không yêu cầu xây dựng hệ thống bệnh án điện tử đầy đủ hoặc quản lý toàn bộ bệnh viện.

Đề tài 6. Hệ thống bán hàng tại cửa hàng

Một cửa hàng bán lẻ muốn xây dựng hệ thống hỗ trợ hoạt động bán hàng tại quầy.

Nhân viên sử dụng hệ thống để tìm kiếm sản phẩm, lập giao dịch bán hàng và thực hiện thanh toán cho khách. Cửa hàng đồng thời cần theo dõi hàng hóa, giá bán và tình trạng tồn kho.

Trong quá trình bán hàng có thể xảy ra những trường hợp như sản phẩm hết hàng, khách thay đổi lựa chọn, áp dụng chương trình giảm giá hoặc giao dịch cần được hủy hoặc điều chỉnh.

Phạm vi: tập trung vào quy trình bán hàng và quản lý hàng hóa liên quan trực tiếp đến bán hàng; không cần xây dựng toàn bộ hệ thống quản trị doanh nghiệp.

Đề tài 7. Hệ thống quản lý ký túc xá

Một trường đại học quản lý khu ký túc xá dành cho sinh viên.

Sinh viên có nhu cầu đăng ký chỗ ở và được bố trí vào các phòng phù hợp. Ban quản lý cần theo dõi tình trạng phòng, người đang ở, thời gian lưu trú và các khoản phí liên quan.

Trong thời gian ở, sinh viên có thể thay đổi phòng, gia hạn hoặc kết thúc việc ở. Một số trường hợp đặc biệt có thể làm thay đổi việc bố trí phòng hoặc tình trạng sử dụng phòng.

Phạm vi: tập trung vào đăng ký, bố trí phòng, quản lý lưu trú và các khoản phí liên quan; không cần xây dựng hệ thống quản lý toàn bộ hoạt động của trường.

Đề tài 8. Hệ thống quản lý cửa hàng điện thoại

Một cửa hàng kinh doanh điện thoại và phụ kiện cần hệ thống hỗ trợ quản lý sản phẩm và bán hàng.

Mỗi sản phẩm có các thông tin và đặc điểm riêng cần được quản lý. Cửa hàng cần theo dõi hàng nhập, hàng tồn, quá trình bán hàng và thông tin khách hàng khi cần thiết.

Sau khi mua hàng, khách có thể phát sinh nhu cầu đổi sản phẩm, bảo hành hoặc xử lý các vấn đề liên quan đến sản phẩm đã mua.

Phạm vi: tập trung vào sản phẩm, tồn kho, bán hàng và các hoạt động sau bán hàng như đổi trả hoặc bảo hành. Không cần mô hình hóa toàn bộ chuỗi cung ứng.

Đề tài 9. Hệ thống quản lý công việc của nhóm

Một nhóm phát triển phần mềm có nhiều thành viên cùng thực hiện các công việc trong một hoặc nhiều dự án.

Nhóm cần theo dõi dự án, công việc, người thực hiện và tiến độ. Công việc có thể được giao, thực hiện, cập nhật và hoàn thành trong quá trình làm việc.

Trong thực tế, một công việc có thể được thay đổi người thực hiện, điều chỉnh mức độ ưu tiên hoặc thay đổi trạng thái. Một số công việc có thể phụ thuộc vào những công việc khác.

Phạm vi: tập trung vào quản lý dự án, công việc, thành viên và tiến độ; không cần xây dựng hệ thống quản lý nhân sự hoặc chấm công.

Đề tài 10. Hệ thống quản lý nhà hàng

Một nhà hàng muốn xây dựng hệ thống hỗ trợ quá trình phục vụ khách từ khi khách đặt bàn hoặc đến nhà hàng cho đến khi hoàn tất thanh toán.

Nhà hàng cần quản lý thông tin bàn, tình trạng sử dụng bàn, thông tin đặt bàn và các món ăn được khách lựa chọn. Trong quá trình phục vụ, nhân viên cần cập nhật tình trạng phục vụ và xử lý các yêu cầu phát sinh của khách.

Khi kết thúc bữa ăn, nhà hàng cần thực hiện thanh toán. Trong thực tế có thể xảy ra nhiều tình huống như thay đổi số lượng khách, thay đổi món, hủy món, tách hoặc gộp thanh toán hoặc thay đổi tình trạng bàn.

Phạm vi: tập trung vào đặt bàn, phục vụ tại bàn, gọi món, quản lý trạng thái bàn và thanh toán; không cần xây dựng toàn bộ hệ thống quản lý nhà hàng.

VI. MỘT SỐ LƯU Ý:

6.1. Yêu cầu chung đối với mọi đề tài

Bất kể lựa chọn đề tài nào, nhóm phải xác định rõ **phạm vi hệ thống** và không mở rộng tùy tiện.

Mỗi nhóm cần bảo đảm:

1. Xác định được các **Actor** và mục tiêu của từng Actor.
2. Xây dựng được các **Use Case chính**.
3. Đặc tả chi tiết một số Use Case tiêu biểu, bao gồm luồng chính và luồng ngoại lệ.
4. Xác định các **Business Rules** quan trọng.
5. Xây dựng **Conceptual Model** phản ánh đúng miền nghiệp vụ.
6. Phân bổ trách nhiệm cho các lớp dựa trên các nguyên tắc thiết kế đã học.
7. Xây dựng **Sequence Diagram** cho các Use Case tiêu biểu.
8. Xây dựng **Design Class Diagram** từ kết quả phân tích.
9. Thiết kế CSDL phù hợp với mô hình và yêu cầu nghiệp vụ.
10. Nếu có cài đặt, code phải phản ánh được thiết kế đã xây dựng.

6.2. Lưu ý về phạm vi

Các phạm vi trên là **mức tham khảo**, không phải danh sách chức năng bắt buộc phải triển khai toàn bộ.

Nhóm có thể thu hẹp hoặc mở rộng phạm vi sau khi được giảng viên phê duyệt, nhưng phải bảo đảm:

- Không quá đơn giản đến mức chỉ còn các chức năng CRUD.
- Không quá lớn so với thời gian và nhân lực.
- Có đủ nghiệp vụ để thể hiện năng lực phân tích và thiết kế hướng đối tượng.
- Không tạo thêm lớp hoặc bảng chỉ nhằm làm tăng số lượng.
- Mọi thành phần quan trọng trong mô hình phải có lý do nghiệp vụ hoặc lý do thiết kế rõ ràng.

Một thiết kế tốt không phải là thiết kế có nhiều lớp, nhiều bảng hoặc nhiều biểu đồ nhất; mà là thiết kế giải quyết đúng bài toán với trách nhiệm được phân bổ hợp lý, phụ thuộc được kiểm soát và có khả năng thích ứng với thay đổi.

6.3. Một số lưu ý khác:

1. Các nhóm có thể chọn một trong các đề tài trên, hoặc đề xuất một đề tài khác với phạm vi tương đương.
2. Các nhóm có thể sử dụng AI để hỗ trợ viết code mẫu, nhưng phải tự kiểm chứng và điều chỉnh thiết kế.
3. Dùng công cụ: Tuyệt đối không vẽ tay. Hãy dùng StarUML, Visual Paradigm, Mermaid… để quản lý mô hình. Lưu file thiết kế để dễ chỉnh sửa.
4. Đừng "Over-engineering": Không nên tạo quá nhiều lớp. Một lớp tốt phải phản ánh đúng bài toán (Problem domain). Nếu một lớp chỉ có 1-2 phương thức đơn giản, hãy cân nhắc gộp vào lớp khác.
5. Mỗi sơ đồ cần thể hiện rõ ý nghĩa:

Diagram	**Câu hỏi cần trả lời**
Use Case	Hệ thống phải cung cấp giá trị gì cho ai?
Use Case Description	Một nghiệp vụ thực sự diễn ra như thế nào?
Conceptual Model	Thế giới nghiệp vụ gồm những đối tượng nào?

Diagram	Câu hỏi cần trả lời
System Sequence Diagram	Actor gửi yêu cầu gì cho hệ thống?
Sequence	Các đối tượng phối hợp thế nào để thực hiện UC?
Design Class Diagram	Ta sẽ xây dựng các lớp cụ thể ra sao?
State Machine	Đối tượng thay đổi trạng thái như thế nào?
ERD	Dữ liệu được lưu trữ thế nào?